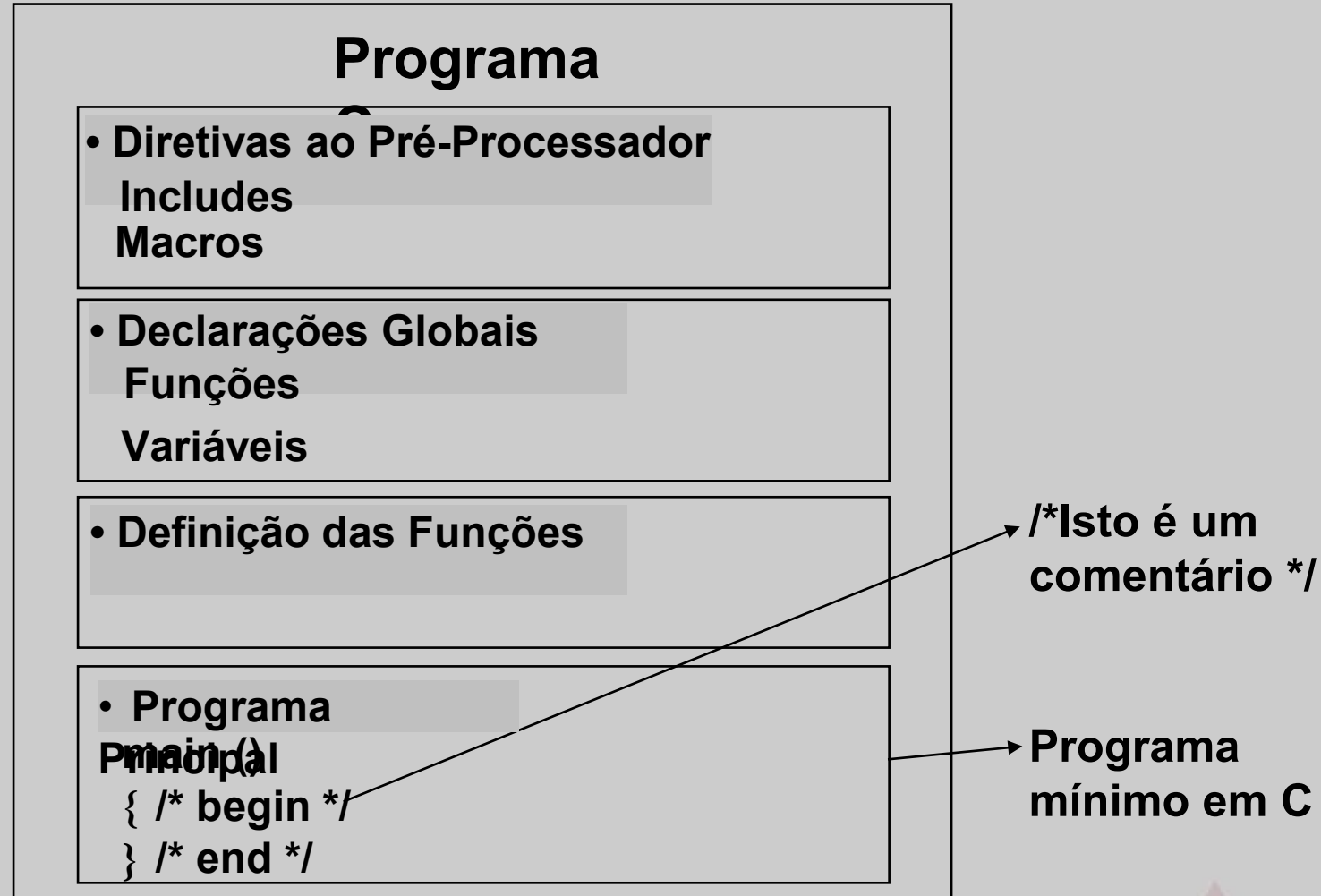


SISTEMAS COMPUTACIONAIS

**Linguagem C: ponteiro,
endereço e memória**

PROGRAMAS C



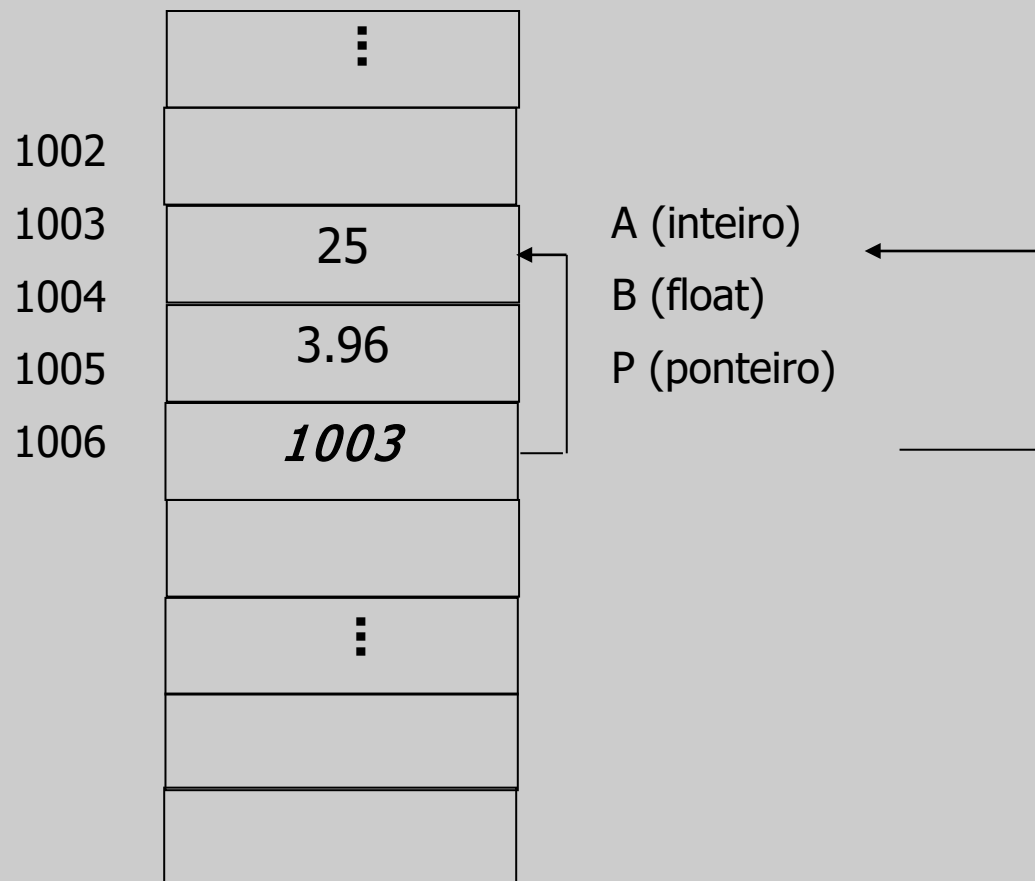
PONTEIROS

- O que é?
 - Variável que contém um endereço de memória, que é a posição de outra variável na memória
 - Dizemos que um ponteiro “aponta” para uma variável

PONTEIROS

- **Por quê?**
 - **São usados para alocar memória dinamicamente**
 - **Pode aumentar a eficiência de certas rotinas**
 - **Ponteiros fornecem meios para funções modificarem seus argumentos**

EXEMPLO



EXEMPLO

Forma geral: tipo * identificador;

tipo: qualquer tipo válido em C

identificador: qualquer identificador válido em C

***: símbolo para declaração de ponteiro. Indica que o identificador aponta para uma variável do tipo C**

Exemplo:

int *p; char *p1; float *pf1, *pf2;

OPERADORES DE PONTEIROS

&

Precedência:

Maior precedência

() [] ++ (pós) -- (pós)

! ~ ++(pré) -- (pré) - (unário) (cast) *(unário) &(unário) sizeof

*** / %**

+ -

<< >>

...

Menor precedência

;

O OPERADOR &

&: operador unário. Devolve o endereço de memória do seu operando. Seu uso mais comum é durante inicializações de ponteiros

Ex.:

```
int *p, acm = 35;
```

```
p = &acm; // p recebe “o endereço de” acm
```

O valor de p é 1292 e o conteúdo é o valor do endereço apontado por p. Outro modo de inicializar:

```
p = 0;
```

```
p = NULL; /*equivale a p = 0*/
```

	·	:	
1002			
1003	1292		p
1004			
	·	:	
1292	35		ACM
1293			
	·	:	

O OPERADOR *

***: operador unário. Devolve o valor da variável apontada (o conteúdo do apontador)**

Ex.:

```
int *p, q, acm = 35;  
p = &acm;  
q = *p;    /* a variável q recebe o  
valor da variável “no endereço” p */
```

O valor de q é 35 (valor da variável apontada por p)

	·	:	
1002			
1003	1292		p
1004			
	·	:	
1292	35		ACM
1293			
	·	:	

ATENÇÃO!!

- Não confundir os operadores de ponteiros & (“o endereço de”) e * (“no endereço”) com o operador AND bit a bit e com o sinal de multiplicação
- Os operadores de ponteiros têm precedência maior que todos os operadores aritméticos, exceto o menos unário

ATENÇÃO!!

- Tecnicamente, qualquer ponteiro pode apontar para qualquer posição de memória
- O tipo do ponteiro pode ser importante. Por exemplo, em aritmética de ponteiros
- `float x = 3.5, y = 0.96;`
- `int *p;`
- `p = &x; /* p (int *) aponta p/ um float!!!*/`
- `y = *p; /*erro*/`

EXPRESSÕES COM PONTEIROS

- **Somente 3 operações são possíveis:**
 - **Atribuição de ponteiros**
 - **Aritmética de ponteiros**
 - **Comparação de ponteiros**

ATRIBUIÇÃO

- Igual a qualquer variável
- Exemplo:

```
int x=8, *p1, *p2;
```

```
p1 = &x;
```

```
p2 = p1;
```

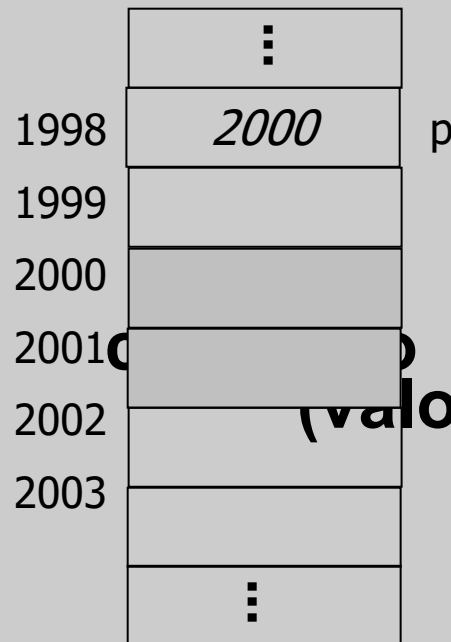
```
printf ("%d %d", p1, *p1);
```

```
printf ("%d %d", p2, *p2);
```

ARITMÉTICA DE PONTEIROS

- **2 operações apenas: adição e subtração**
 - **Cada incremento (ou decremento) caminha para frente (ou retrocede) o ponteiro na quantidade de bytes correspondente ao tipo base (incrementa o endereço do ponteiro)**

ARITMÉTICA DE PONTEIROS



(valor) armazenado em &p

inteiro de 2 bytes

```
int *p;
```

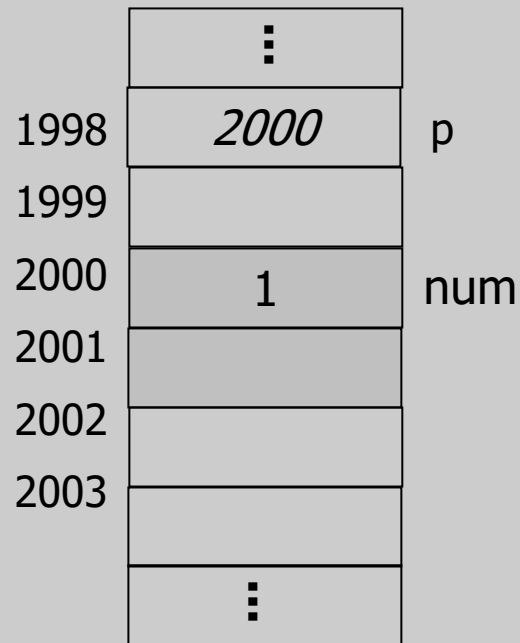
...

```
p++; //incrementa
```

```
/*p--;*/
```

```
p = p + 2; /*válido*/
```

ARITMÉTICA DE PONTEIROS



num2?

inteiro de 2 bytes:

```
int *p, num=1, num2;
```

...

```
p = &num;
```

```
num = *p+1;
```

```
num2 = *&num;
```

Quanto vale num? E

COMPARAÇÃO DE PONTEIROS

- Comparação entre ponteiros é possível

- Compara-se as posições de memória

- Exemplo:

```
int *p1, *p2, x =10;
```

```
p1 = &x;
```

```
p2 = p1;
```

```
if (p1 == p2) ...
```

ou

```
if (p1) ... /* p == 0: ponteiro nulo*/
```

BIBLIOGRAFIA

- **SCHILDT, H., C Completo e Total, Makron Books, 1997. – Capítulo 2.**
- **KELLEY, A.; POHL, I., A Book on C, Addison-Wesley, 4a edição, 1998. – Capítulos 1, 2 e 4.**
- **Damas, L. Linguagem C. Editora Grupo GEN. 10/2006. VBID 9788521632474**
- **Soffner, R. Algoritmos e Programação em Linguagem C, 1ª edição. Editora Saraiva. 08/2013. VBID 9788502207530**

SISTEMAS COMPUTACIONAIS

**Linguagem C: ponteiro,
endereço e memória**